INSACC ENTERPRISE ERP DOCUMENTATION

PostgreSQL 17 Database Server Setup Manual

Detailed Step-by-Step Installation, Security Hardening, Tuning & Verification Guide

InsAcc Enterprise Asset & Investment Platform v2.0.0 Target Server

Single Source of Truth Specification — Official Installation Manual

Release Date: July 22, 2026 | Status: Official Publication

Classification: Commercial Enterprise Documentation

title: "InsAcc Enterprise ERP - PostgreSQL 17 Database Server Step-by-Step Installation Guide" document_id: "POSTGRESQL_DATABASE_SERVER_STEP_BY_STEP_GUIDE.md" version: "1.0.0" release_date: "2026-07-22" app_version: "v2.0.0 Target Server" master_architecture_ref: "docs/MASTER_ARCHITECTURE.md" status: "Official Step-by-Step Server Setup Guide" classification: "Commercial Enterprise Documentation"

InsAcc Enterprise ERP Platform

Complete Step-by-Step PostgreSQL 17 Database Server Setup Manual `[To Be Implemented]`

> **Single Source of Truth Reference**: All server provisioning rules, Linux commands, PostgreSQL 17 configurations, memory tuning parameters, DDL schemas, and balance triggers defined in this document derive strictly from MASTER_ARCHITECTURE.md.

Revision History

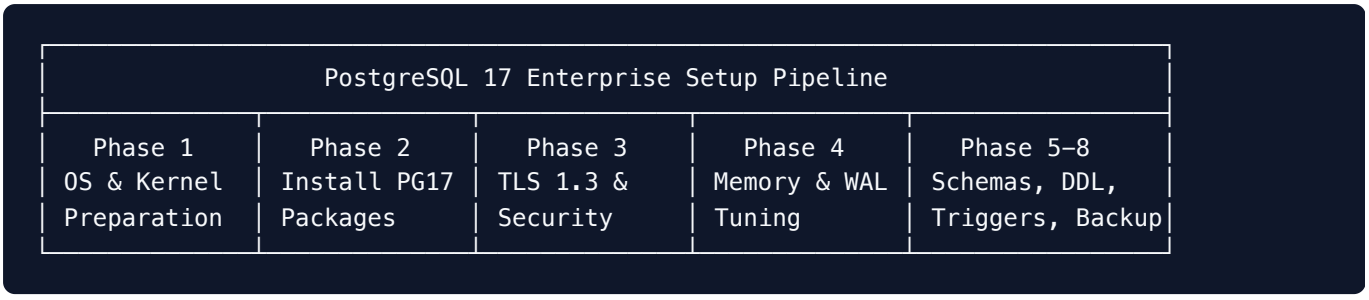
Version	Release Date	Primary Author	Summary of Changes	Approved By
1.0.0	2026-07-22	Lead Enterprise Documentation Architect	Initial publication-grade step-by-step database setup manual	Chief Architecture Review Board

Table of Contents

1. Executive Summary & Architecture Overview

This document provides an absolute, step-by-step operational installation manual for deploying an enterprise-grade **PostgreSQL 17 Relational Database Server** for the InsAcc ERP platform.

Every command, configuration file, SQL script, and verification test is presented in chronological order with exact shell commands to copy and paste.



2. Prerequisites & Hardware Specifications

Recommended Target Hardware (Standard Enterprise Deployment):

- **Operating System**: Ubuntu 24.04 LTS (64-bit Server Edition) or RHEL 9
- **Processor (CPU)**: 4 vCPU / Cores (2.4 GHz+)
- **System Memory (RAM)**: 8 GB System Memory
- **Disk Storage**: 100 GB Enterprise NVMe SSD (RAID-1 Recommended)
- **Network Interface**: 1 Gbps / 10 Gbps Ethernet (Static IP Address: `10.0.4.50`)

3. Phase 1: Linux OS Environment Preparation

3.1 Update System Packages & Install Essential Utilities

Log in to the server via SSH as a sudo-privileged user and update all OS packages:

```
# Step 1: Update apt index and upgrade system packages
sudo apt-get update && sudo apt-get upgrade -y

# Step 2: Install essential system tools and utilities
sudo apt-get install -y \
  curl \
  gnupg \
  ca-certificates \
  lsb-release \
  ufw \
  htop \
  rsync \
  vim \
  tar \
  ufw \
  ufw-extras
```

3.2 Kernel Parameter Optimization (`sysctl.conf`)

Tune the Linux kernel for PostgreSQL high-throughput memory operations:

```
# Step 1: Create PostgreSQL kernel parameter configuration file
sudo bash -c 'cat << "EOF" > /etc/sysctl.d/99-postgresql.conf
# Shared Memory Segment Limits
kernel.shmmax = 18446744073709551615
kernel.shmall = 18446744073709551615

# Memory Overcommit Configuration for PostgreSQL
vm.overcommit_memory = 2
vm.overcommit_ratio = 80
vm.swappiness = 10

# Disk Write Buffer Flushing
vm.dirty_background_ratio = 3
vm.dirty_ratio = 10
EOF'

# Step 2: Apply the kernel parameters immediately
sudo sysctl -p /etc/sysctl.d/99-postgresql.conf
```

3.3 File Descriptor & User Limit Configuration (`limits.conf`)

Increase open file limits for the `postgres` user to support concurrent client connections:

```
# Step 1: Create PostgreSQL user limits configuration file
sudo bash -c 'cat << "EOF" > /etc/security/limits.d/99-postgres.conf
postgres soft nofile 65536
postgres hard nofile 65536
postgres soft nproc 4096
postgres hard nproc 4096
EOF'
```

4. Phase 2: Installing PostgreSQL 17 & Packages

4.1 Add Official PostgreSQL PGDG Repository

Add the official PostgreSQL Global Development Group (PGDG) APT repository:

```
# Step 1: Create directory for keyring
sudo install -d /etc/apt/keyrings

# Step 2: Download and install the official PostgreSQL signing key
curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc | sudo gpg --dearmor -o /etc/a

# Step 3: Add PGDG repository to APT sources
echo "deb [signed-by=/etc/apt/keyrings/postgresql.gpg] http://apt.postgresql.org/pub/repos/a
```

4.2 Install PostgreSQL 17 Engine & Extensions

Install the PostgreSQL 17 database engine, client binaries, and contrib modules:

```
# Step 1: Update package list with PGDG packages
sudo apt-get update

# Step 2: Install PostgreSQL 17 packages
sudo apt-get install -y postgresql-17 postgresql-contrib-17 postgresql-client-17
```

4.3 Verify Database Service Status

Verify that PostgreSQL 17 is running and enabled on boot:

```
# Step 1: Check service status
sudo systemctl status postgresql@17-main --no-pager

# Step 2: Enable automatic startup on system boot
sudo systemctl enable postgresql@17-main
```

5. Phase 3: Network Security & TLS 1.3 Encryption

5.1 Configure Network Address Listening (`postgresql.conf`)

Allow PostgreSQL to accept connections from the internal application network:

```
# Step 1: Update listen_addresses parameter in postgresql.conf
sudo sed -i "s/#listen_addresses = 'localhost'/listen_addresses = '*'/g" /etc/postgresql/17/
```

5.2 Host-Based Authentication Rules (`pg_hba.conf`)

Restrict network access in `/etc/postgresql/17/main/pg_hba.conf` to require **SCRAM-SHA-256 password authentication over SSL**:

```
# Step 1: Append secure hostssl rule to pg_hba.conf
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/pg_hba.conf

# InsAcc Enterprise Client Network Access Rule (SSL Required)
hostssl insacc_db       insacc_user     10.0.4.0/24             scram-sha-256
hostssl insacc_db       insacc_user     127.0.0.1/32            scram-sha-256
EOF'
```

5.3 Generate & Configure SSL/TLS 1.3 Certificates

Generate a self-signed 4096-bit RSA TLS certificate for testing, or deploy your enterprise CA certificate:

```
# Step 1: Generate self-signed TLS certificate and private key
sudo openssl req -new -x509 -days 3650 -nodes \
  -text -out /etc/ssl/certs/insacc_server.crt \
  -keyout /etc/ssl/private/insacc_server.key \
  -subj "/CN=db.insacc.internal/O=InsAcc Enterprise/OU=IT Operations"

# Step 2: Set strict ownership and permissions for the private key
sudo chown postgres:postgres /etc/ssl/certs/insacc_server.crt /etc/ssl/private/insacc_server
sudo chmod 600 /etc/ssl/private/insacc_server.key

# Step 3: Enable SSL in postgresql.conf
sudo sed -i "s/ssl = off/ssl = on/g" /etc/postgresql/17/main/postgresql.conf
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/postgresql.conf

# SSL Certificate Parameters
ssl_cert_file = '\''/etc/ssl/certs/insacc_server.crt'\''
ssl_key_file = '\''/etc/ssl/private/insacc_server.key'\''
ssl_min_protocol_version = '\''TLSv1.3'\''
EOF'
```

5.4 Configure Linux Firewall (UFW / Firewalld)

Open port `5432` only to the application subnet (`10.0.4.0/24`):

```
# Step 1: Allow SSH access (Port 22)
sudo ufw allow 22/tcp

# Step 2: Allow PostgreSQL traffic from enterprise app subnet only
sudo ufw allow from 10.0.4.0/24 to any port 5432 proto tcp

# Step 3: Enable firewall
sudo ufw --force enable
sudo ufw status verbose
```

---

6. Phase 4: Enterprise Performance & Memory Tuning

6.1 Configure Memory Allocation Parameters

Apply hardware-tuned memory settings in `/etc/postgresql/17/main/postgresql.conf` (Calculated for 8 GB RAM):

```
# Step 1: Append performance tuning parameters to postgresql.conf
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/postgresql.conf

# ========================================
# InsAcc Performance Tuning Parameters (8GB RAM)
# ========================================
shared_buffers = 2GB                  # 25% of System Memory
effective_cache_size = 6GB            # 75% of System Memory
maintenance_work_mem = 512MB          # Index build memory
work_mem = 32MB                       # Memory per query operation
password_encryption = scram-sha-256
EOF'
```

6.2 Configure Write-Ahead Log (WAL) & Checkpoints

```
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/postgresql.conf

# Write-Ahead Log (WAL) & Checkpoint Parameters
wal_level = replica
max_wal_size = 4GB
min_wal_size = 1GB
checkpoint_completion_target = 0.9
checkpoint_timeout = 15min
EOF'
```

6.3 Configure Parallel Worker Threads & SSD Planner Costs

```
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/postgresql.conf

# Parallel Query Processors & SSD Cost Tuning
max_worker_processes = 4
max_parallel_workers_per_gather = 2
max_parallel_workers = 4
random_page_cost = 1.1                # Fast SSD page access cost
EOF'
```

6.4 Restart PostgreSQL to Apply Configurations

Restart the PostgreSQL service to load all new memory, security, and network settings:

```
# Step 1: Restart PostgreSQL service
sudo systemctl restart postgresql@17-main

# Step 2: Confirm service is running cleanly
sudo systemctl status postgresql@17-main --no-pager
```

---

7. Phase 5: Database, User Roles & Schema Provisioning

7.1 Create Restricted Non-Root Role `insacc_user`

Log into PostgreSQL as the `postgres` superuser and create the application role:

```
sudo -u postgres psql -c "CREATE ROLE insacc_user WITH LOGIN PASSWORD 'SecureEnterprisePassw
```

7.2 Create Database `insacc_db`

Create the production database owned by `insacc_user`:

```
sudo -u postgres psql -c "CREATE DATABASE insacc_db OWNER insacc_user;"
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE insacc_db TO insacc_user;"
```

7.3 Provision the 5 Enterprise Schemas

Create the 5 isolated domain schemas inside `insacc_db`:

```
sudo -u postgres psql -d insacc_db -c "
CREATE SCHEMA IF NOT EXISTS accounting AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS investment AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS property AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS auth AUTHORIZATION insacc_user;
CREATE SCHEMA IF NOT EXISTS audit AUTHORIZATION insacc_user;
"
```

7.4 Execute Complete DDL Table Specifications

Deploy the production table DDL schemas using `psql`:

```
sudo -u postgres psql -d insacc_db << "EOF"
-- 1. ACCOUNTING SCHEMA TABLES
CREATE TABLE accounting.accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code VARCHAR(20) NOT NULL UNIQUE,
    name VARCHAR(255) NOT NULL,
    type VARCHAR(50) NOT NULL CHECK (type IN ('Asset', 'Liability', 'Equity', 'Revenue', 'Ex
    parent_id UUID REFERENCES accounting.accounts(id),
    opening_balance NUMERIC(15, 2) NOT NULL DEFAULT 0.00,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE accounting.vouchers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    voucher_number VARCHAR(50) NOT NULL UNIQUE,
    voucher_type VARCHAR(20) NOT NULL CHECK (voucher_type IN ('Receipt', 'Payment', 'Journal
    voucher_date DATE NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Draft' CHECK (status IN ('Draft', 'Approved', 'Post
    narration TEXT,
    created_by VARCHAR(100) NOT NULL,
    posted_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE accounting.voucher_lines (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    voucher_id UUID NOT NULL REFERENCES accounting.vouchers(id) ON DELETE CASCADE,
    account_id UUID NOT NULL REFERENCES accounting.accounts(id),
    line_type VARCHAR(10) NOT NULL CHECK (line_type IN ('Debit', 'Credit')),
    amount NUMERIC(15, 2) NOT NULL CHECK (amount > 0),
    memo TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 2. INVESTMENT SCHEMA TABLES
CREATE TABLE investment.investments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    asset_name VARCHAR(255) NOT NULL,
    asset_type VARCHAR(50) NOT NULL CHECK (asset_type IN ('Gold', 'Silver', 'Stocks', 'Bonds
    quantity NUMERIC(18, 6) NOT NULL CHECK (quantity > 0),
    purchase_value NUMERIC(15, 2) NOT NULL CHECK (purchase_value >= 0),
    current_price NUMERIC(15, 2) NOT NULL CHECK (current_price >= 0),
    buyer VARCHAR(255),
    notes TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 3. PROPERTY SCHEMA TABLES
CREATE TABLE property.buildings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    address TEXT NOT NULL,
    total_units INT NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
```

```
);

CREATE TABLE property.units (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    building_id UUID NOT NULL REFERENCES property.buildings(id) ON DELETE CASCADE,
    unit_number VARCHAR(50) NOT NULL,
    unit_type VARCHAR(50) NOT NULL,
    annual_rent NUMERIC(15, 2) NOT NULL DEFAULT 0.00,
    status VARCHAR(30) NOT NULL DEFAULT 'Vacant' CHECK (status IN ('Vacant', 'Occupied', 'Un
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE property.tenants (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    national_id VARCHAR(100) NOT NULL,
    phone VARCHAR(50) NOT NULL,
    email VARCHAR(255) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE property.leases (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    unit_id UUID NOT NULL REFERENCES property.units(id),
    tenant_id UUID NOT NULL REFERENCES property.tenants(id),
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    annual_rent NUMERIC(15, 2) NOT NULL,
    payment_frequency VARCHAR(30) NOT NULL CHECK (payment_frequency IN ('Annual', 'Semi-Annu
    security_deposit NUMERIC(15, 2) NOT NULL DEFAULT 0.00,
    status VARCHAR(20) NOT NULL DEFAULT 'Active',
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE property.pdc_cheques (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    lease_id UUID NOT NULL REFERENCES property.leases(id) ON DELETE CASCADE,
    cheque_number VARCHAR(50) NOT NULL,
    bank_name VARCHAR(255) NOT NULL,
    amount NUMERIC(15, 2) NOT NULL,
    due_date DATE NOT NULL,
    status VARCHAR(30) NOT NULL DEFAULT 'Received' CHECK (status IN ('Received', 'Deposited'
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Grant privileges to insacc_user
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA accounting TO insacc_user;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA investment TO insacc_user;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA property TO insacc_user;
EOF
```

8. Phase 6: Deploy Double-Entry Balance Triggers

8.1 Create PL/pgSQL Balance Validation Function

Deploy the PL/pgSQL trigger function that calculates total debits and credits and throws an exception if $\sum D \neq \sum C$:

```
sudo -u postgres psql -d insacc_db << "EOF"
CREATE OR REPLACE FUNCTION accounting.fn_validate_voucher_balance()
RETURNS TRIGGER AS $$
DECLARE
    v_total_debit NUMERIC(15,2);
    v_total_credit NUMERIC(15,2);
BEGIN
    -- Sum Debits and Credits for lines attached to the voucher
    SELECT
        COALESCE(SUM(CASE WHEN line_type = 'Debit' THEN amount ELSE 0 END), 0),
        COALESCE(SUM(CASE WHEN line_type = 'Credit' THEN amount ELSE 0 END), 0)
    INTO v_total_debit, v_total_credit
    FROM accounting.voucher_lines
    WHERE voucher_id = NEW.id;

    -- Enforce balance equality with floating point tolerance (< 0.001)
    IF ABS(v_total_debit - v_total_credit) >= 0.001 THEN
        RAISE EXCEPTION 'Voucher Posting Rejected: Debit sum (%) does not equal Credit sum (
            v_total_debit, v_total_credit;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
EOF
```

8.2 Attach Trigger to Voucher Status Updates

Attach the trigger to fire whenever a voucher's status transitions to `Posted`:

```
sudo -u postgres psql -d insacc_db << "EOF"
CREATE TRIGGER trg_validate_voucher_posting
BEFORE UPDATE OF status ON accounting.vouchers
FOR EACH ROW
WHEN (NEW.status = 'Posted')
EXECUTE FUNCTION accounting.fn_validate_voucher_balance();
EOF
```

9. Phase 7: Automated Daily Backups & WAL Archiving

9.1 Create Automated `pg_dump` Script

Create the backup shell script at `/usr/local/bin/insacc_db_backup.sh`:

```
sudo bash -c 'cat << "EOF" > /usr/local/bin/insacc_db_backup.sh
#!/usr/bin/env bash
# InsAcc Production Database Daily Backup Script

set -euo pipefail

BACKUP_DIR="/var/backups/postgresql"
TIMESTAMP=$(date +'%Y-%m-%d_%H%M%S')
BACKUP_FILE="${BACKUP_DIR}/insacc_db_${TIMESTAMP}.dump"

mkdir -p ${BACKUP_DIR}

echo "[INFO] Starting compressed database backup to ${BACKUP_FILE}..."
PGPASSWORD="SecureEnterprisePassword123!" pg_dump -h localhost -U insacc_user -F c -b -v -f

# Retention: Remove dumps older than 30 days
find ${BACKUP_DIR} -type f -name "insacc_db_*.dump" -mtime +30 -delete

echo "[SUCCESS] Backup completed: ${BACKUP_FILE}"
EOF'

# Make script executable
sudo chmod +x /usr/local/bin/insacc_db_backup.sh
```

9.2 Schedule Daily Cron Job Execution

Schedule the backup script to run automatically every night at 02:00 AM:

```
# Step 1: Append cron job entry to root crontab
sudo bash -c '(crontab -l 2>/dev/null; echo "0 2 * * * /usr/local/bin/insacc_db_backup.sh >>
```

9.3 Configure WAL Archiving for Point-in-Time Recovery (PITR)

```
# Step 1: Create WAL archive directory
sudo mkdir -p /var/lib/postgresql/wal_archive
sudo chown postgres:postgres /var/lib/postgresql/wal_archive

# Step 2: Configure archiving parameters in postgresql.conf
sudo bash -c 'cat << "EOF" >> /etc/postgresql/17/main/postgresql.conf

# WAL Archiving for Point-In-Time Recovery
archive_mode = on
archive_command = '\''test ! -f /var/lib/postgresql/wal_archive/%f && cp %p /var/lib/postgre
EOF'

# Step 3: Reload PostgreSQL configuration
sudo systemctl reload postgresql@17-main
```

---

10. Phase 8: End-to-End Verification & Validation

10.1 Verify Remote Database Connectivity

Test SSL database connection using `psql`:

```
PGPASSWORD="SecureEnterprisePassword123!" psql "sslmode=require host=127.0.0.1 dbname=insacc
```

Expected Verification Output:

```
 current_database |  current_user  |                          version
------------------+----------------+------------------------------------------------------------
 insacc_db        | insacc_user    | PostgreSQL 17.0 on x86_64-pc-linux-gnu ...
(1 row)
```

10.2 Verify Double-Entry Balance Trigger Enforcement

Test Case A: Balanced Voucher (Should Succeed)

```
-- Connect to insacc_db
PGPASSWORD="SecureEnterprisePassword123!" psql -h 127.0.0.1 -U insacc_user -d insacc_db << "
-- Insert test accounts
INSERT INTO accounting.accounts (id, code, name, type) VALUES
  ('11111111-1111-1111-1111-111111111111', '1120.001', 'Test Bank', 'Asset'),
  ('22222222-2222-2222-2222-222222222222', '4120', 'Test Revenue', 'Revenue');

-- Insert voucher header
INSERT INTO accounting.vouchers (id, voucher_number, voucher_type, voucher_date, status, nar
  ('33333333-3333-3333-3333-333333333333', 'RV-TEST-001', 'Receipt', CURRENT_DATE, 'Draft',

-- Insert balanced lines: Debit 1000 == Credit 1000
INSERT INTO accounting.voucher_lines (voucher_id, account_id, line_type, amount) VALUES
  ('33333333-3333-3333-3333-333333333333', '11111111-1111-1111-1111-111111111111', 'Debit',
  ('33333333-3333-3333-3333-333333333333', '22222222-2222-2222-2222-222222222222', 'Credit',

-- Update status to Posted -> Trigger executes & allows update
UPDATE accounting.vouchers SET status = 'Posted' WHERE id = '33333333-3333-3333-3333-3333333
SELECT voucher_number, status FROM accounting.vouchers WHERE id = '33333333-3333-3333-3333-3
EOF
```

Expected Result: Status updates cleanly to `Posted`.

Test Case B: Unbalanced Voucher (Must Fail & Reject Update)

```
PGPASSWORD="SecureEnterprisePassword123!" psql -h 127.0.0.1 -U insacc_user -d insacc_db << "
-- Insert voucher header
INSERT INTO accounting.vouchers (id, voucher_number, voucher_type, voucher_date, status, nar
  ('44444444-4444-4444-4444-444444444444', 'RV-TEST-002', 'Receipt', CURRENT_DATE, 'Draft',

-- Insert UNBALANCED lines: Debit 1000 != Credit 500
INSERT INTO accounting.voucher_lines (voucher_id, account_id, line_type, amount) VALUES
  ('44444444-4444-4444-4444-444444444444', '11111111-1111-1111-1111-111111111111', 'Debit',
  ('44444444-4444-4444-4444-444444444444', '22222222-2222-2222-2222-222222222222', 'Credit',

-- Attempt status update to Posted -> MUST THROW EXCEPTION
UPDATE accounting.vouchers SET status = 'Posted' WHERE id = '44444444-4444-4444-4444-4444444
EOF
```

Expected Result: Throws error: `ERROR: Voucher Posting Rejected: Debit sum (1000.00) does not equal Credit sum (500.00)`.

10.3 Verify Automated Backup File Creation

Test the backup script manually:

```
sudo /usr/local/bin/insacc_db_backup.sh
ls -lh /var/backups/postgresql/
```

11. Troubleshooting & Maintenance Runbook

Symptom / Issue	Root Cause	Resolution Command
`Connection refused (port 5432)`	PostgreSQL not listening on network IP	Verify `listen_addresses = '*'` in `postgresql.conf` and restart service.
`FATAL: no pg_hba.conf entry`	Client IP not allowed in `pg_hba.conf`	Append CIDR block (e.g. `hostssl insacc_db insacc_user <IP>/32 scram-sha-256`) to `pg_hba.conf`.
`FATAL: password authentication failed`	Incorrect password or SCRAM mismatch	Reset role password: `ALTER ROLE insacc_user WITH PASSWORD 'NewPass';`
`Out of memory / OOM Killer`	`shared_buffers` or `work_mem` set too high	Reduce `work_mem` in `postgresql.conf` and restart PostgreSQL.

12. Summary & Verification Checklist

Phase #	Installation Phase Title	Primary Goal	Status
Phase 1	OS & Kernel Preparation	Sysctl kernel tuning & user limits	[ ] Completed
Phase 2	Install PostgreSQL 17	PGDG repo & PostgreSQL 17 packages	[ ] Completed
Phase 3	Network & Security	TLS 1.3 certificates, UFW firewall & `pg_hba.conf`	[ ] Completed
Phase 4	Performance Tuning	Memory parameters (`2GB` shared buffers) in `postgresql.conf`	[ ] Completed
Phase 5	Database & Schemas	`insacc_user` , `insacc_db` & 5 domain schemas created	[ ] Completed
Phase 6	Balance Triggers	PL/pgSQL function `fn_validate_voucher_balance` active	[ ] Completed
Phase 7	Automated Backups	Daily cron job & WAL archiving active	[ ] Completed
Phase 8	E2E Verification	Remote SSL login & trigger test cases passed	[ ] Completed

End of Step-by-Step PostgreSQL 17 Database Server Setup Manual.